

ROLL NUMBER: 240701350 | NAME: NAVEEN RAJ S

Experiment - 2

Comparative Study of Low-Code Development versus Traditional Application Development

Aim

To study and compare low-code development with traditional application development, looking at their stages, workflow, and the technical differences between the two.

Introduction to Traditional Approach

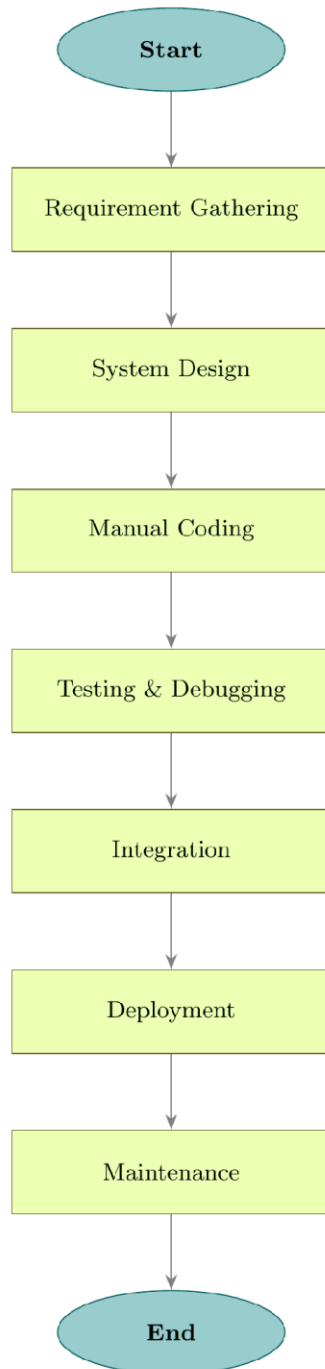
Traditional software development is built around manual, line-by-line coding using standard editors and IDEs. Developers handle database schemas, API integrations, and server deployment by hand, and the whole process depends heavily on skill in languages like Java, Python, or C++. It gives full control over the codebase and architecture, but usually comes with longer development cycles, higher budgets, and detailed bug testing.

Introduction to Low-Code Approach

Low-Code Application Platforms (LCAPs) cut down on hand-coding by offering cloud-based, visual development environments instead. They rely on drag-and-drop components, pre-built visual models, and automated deployment pipelines, which lowers the learning curve enough that even "citizen developers" with little or no coding background can put together working applications quickly. Since repetitive programming tasks are automated, low-code shortens time-to-market and reduces initial costs, though it can limit deep customization and tie the project to a particular platform.

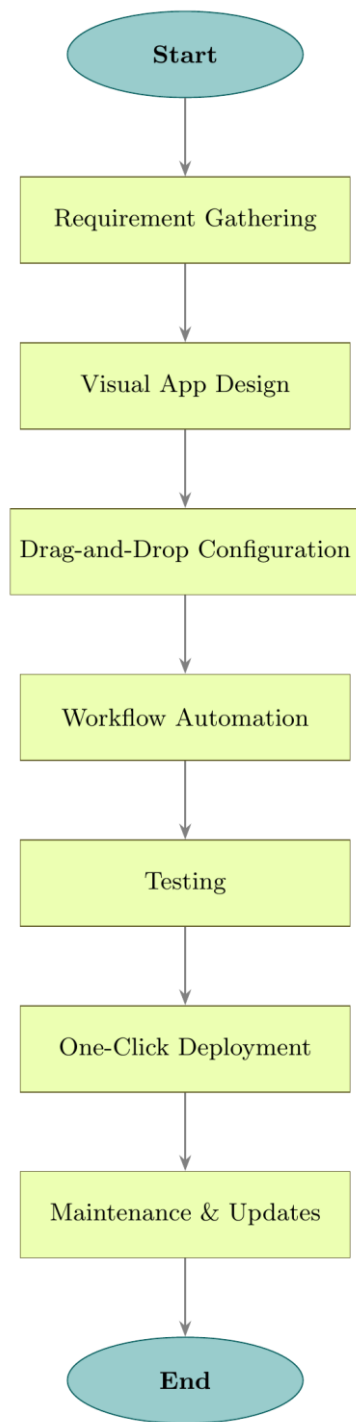
Stages in Traditional Approach

The traditional Software Development Life Cycle (SDLC) follows a linear or iterative sequence, with each stage needing dedicated engineering effort:



Stages in Low Code Approach

The low-code development process replaces manual coding stages with visual, configuration-driven steps that require far less engineering effort at each stage:



Comparative Analysis

Feature	Traditional Development	Low-Code Development
Development Speed	Slow, since everything is coded from scratch by hand	Fast, cuts development time by roughly 50–70% through pre-built modules

Feature	Traditional Development	Low-Code Development
Skill Required	High – needs experienced software engineers	Low – drag-and-drop tools support citizen development
Customization	Full control over every line of code	Limited to what the platform’s components allow
Cost	Higher setup cost, developer salaries, and long-term upkeep	Lower upfront cost, but recurring licensing or subscription fees
Maintenance & Updates	Manual coding and testing needed for every bug fix or update	Updates are handled centrally by the platform provider
Vendor Lock-in	None – business owns the source code fully	High – application depends on the vendor’s ecosystem
Best Use Cases	Complex, high-performance, or highly secure enterprise systems	Prototyping, workflow automation, and internal tools

Result

This comparison shows that traditional development still offers the most flexibility and control, making it suited to mission-critical or highly customized systems, while low-code development gives a faster, cheaper way to build and maintain everyday business applications. In practice, many organizations combine both: using low-code for quick front-end and workflow work, and traditional development for the more complex back-end requirements.